PYTHON MAJOR PROJECT ASSIGNMENT



A Command-Line Turn-Based RPG, Built in Python

"The kingdom of Eldoria has fallen into darkness. Only one hero — and one well-structured Python program — can defeat the Ancient Demon King and bring peace back to the realm."

LANGUAGE	INTERFACE	DIFFICULTY	TOTAL MARKS
Python 3.x	**Command Line (Terminal)**	★★★★☆ **Intermediate–Advanced**	**100**

Table of Contents

1. Objective
2. The Story of Eldoria
3. Character Creation
4. Player Stats
5. Weapon System
6. Armor System
7. Potion System
8. Turn-Based Battle System
9. Critical Hit & Defend Mechanics
10. Class Skills
11. Inventory & Shop
12. Enemy System
13. Boss System
14. Leveling System
15. Loot Drop System
16. World Map
17. Menus, Game Over & Victory Screen
18. Bonus Features (Optional)
19. Technical Requirements
20. Submission Requirements
21. Evaluation Rubric

1 Objective

Design and develop a fully playable **Turn-Based Role-Playing Game (RPG)** using Python. The game must be entirely command-line based and should demonstrate strong Python programming skills across the following areas:

- ✓ Functions
- ✓ Loops
- ✓ Conditional Statements
- ✓ Lists, Dictionaries, Tuples
- ✓ File Handling (JSON / Text)
- ✓ Exception Handling
- ✓ Random Module
- ✓ Modular Programming
- ✓ Object-Oriented Programming (optional)

Students are free to use functions, classes, or a combination of both — the design approach is left open, as long as the final program is clean, modular, and playable start to finish.

2 The Story of Eldoria

The kingdom of Eldoria has fallen into darkness.

Monsters have taken over villages, forests, caves, and ancient ruins.

Every 10 levels, a powerful boss guards the entrance to the next region.

Your mission is to defeat all bosses, reach Level 100, and destroy the Ancient Demon King.

3 Character Creation

When the game starts, the player must be prompted to enter a name and choose a class:

```
Enter Player Name:

Choose Your Class
  1. Warrior
  2. Mage
  3. Archer
  4. Assassin
```

Each class must have unique base stats, reflecting a distinct playstyle:

Class	HP	Mana	Attack	Defense	Crit Chance
Warrior	High	Low	High	High	Low
Mage	Medium	High	High (Magic)	Low	Medium
Archer	Medium	Medium	Medium	Medium	High
Assassin	Low	Medium	High	Low	Very High

4 Player Stats

The program must maintain a persistent record of the player's state, including:

- ✓ Name
- ✓ Class
- ✓ Level
- ✓ Experience
- ✓ HP / Max HP
- ✓ Mana / Max Mana
- ✓ Attack
- ✓ Defense
- ✓ Gold
- ✓ Critical Chance
- ✓ Critical Damage
- ✓ Weapon (equipped)
- ✓ Armor (equipped)
- ✓ Potions
- ✓ Inventory
- ✓ Bosses Defeated
- ✓ Enemies Defeated

5 Weapon System

Implement a weapon rarity system where higher ranks provide stronger base damage and larger critical bonuses.

Rank	Suggested Color	Damage Multiplier	Critical Bonus
Common	White	x1.0	+0%
Uncommon	Green	x1.1	+2%
Rare	Blue	x1.3	+5%
Super Rare	Purple	x1.5	+8%
Epic	Orange	x1.8	+12%
Mythical	Red	x2.2	+18%
Legendary	Gold	x2.8	+25%

Every weapon object/record must contain: **Name, Rank, Damage, Critical Bonus, Price, Required Level.**

Example Weapon Pool (expand freely — minimum 20 unique weapons required)

Rarity	Example Weapons
Common	Wooden Sword, Rusty Axe, Training Bow
Rare	Iron Sword, Steel Axe, Long Bow
Super Rare	Crystal Blade, Shadow Dagger, Lightning Bow
Epic	Flame Sword, Ice Spear, Thunder Hammer
Mythical	Phoenix Blade, Demon Slayer, Celestial Staff
Legendary	Dragon King's Sword, Blade of Eternity, Bow of the Gods

6 Armor System

Armor follows the same six-to-seven tier rarity philosophy as weapons: **Common, Rare, Super Rare, Epic, Mythical, Legendary** (Uncommon may optionally be included for consistency with weapons).

Each armor piece must provide:

- ✓ Defense bonus
- ✓ HP bonus
- ✓ Mana bonus
- ✓ Critical Resistance

A minimum of **12 distinct armor sets** must be implemented across the rarity tiers.

7 Potion System

Health Potions

Potion	HP Restored
Small	50
Medium	100
Large	250
Giant	500
Ultimate	Full HP

Mana Potions

Potion	Mana Restored
Small	30
Medium	75
Large	150
Giant	300
Ultimate	Full Mana

Mixed Potions

Restore both HP and Mana simultaneously in a single use.

Buff Potions

Temporarily increase **Attack**, **Defense**, or **Critical Chance** for 3 turns.

> **Minimum requirement:** at least 15 potion types across health, mana, mixed, and buff categories combined.

8 Turn-Based Battle System

Every turn, the player must be presented with a clear action menu:

```
====================
YOUR TURN
====================
1  Attack
2  Skills
3  Heal
4  Use Potion
5  Defend
6  Inventory
7  View Stats
8  Run
```

After the player's action resolves, the enemy takes its turn automatically (or with simple AI logic based on its HP and available skills).

9 Critical Hit & Defend Mechanics

Critical Hit System

Every attack has a chance to critically strike, based on the character's Critical Chance stat plus any weapon or buff bonuses.

```
Damage = Attack + Weapon Damage − Enemy Defense
Critical Damage = Damage × Critical Multiplier
```

```
CRITICAL HIT!
You dealt 185 Damage!
```

Defend System

If the player selects **Defend**, incoming damage from the enemy's next attack is reduced by **50%**.

10 Class Skills

Every class must have a unique set of skills, each with its own mana cost and effect. Suggested skill sets:

Class	Skills
Warrior	Slash, Shield Bash, Rage, Earthquake
Mage	Fireball, Ice Blast, Thunder Strike, Meteor
Archer	Multi Shot, Poison Arrow, Explosive Arrow, Sniper Shot
Assassin	Backstab, Smoke Bomb, Shadow Strike, Instant Kill (very low chance)

Every skill must consume mana, and higher-tier skills should cost more mana in exchange for stronger effects.

11 Inventory & Shop

Inventory

The inventory system must store weapons, armor, potions, quest items, and materials. The player must be able to:

- ✓ Equip
- ✓ Unequip
- ✓ Use
- ✓ Drop
- ✓ Sell

Shop

The village shop must allow the player to:

- ✓ Buy Weapons
- ✓ Buy Armor
- ✓ Buy Potions
- ✓ Sell Items
- ✓ Exit

Prices should scale with item rarity — higher-rarity gear should cost significantly more gold, and sell for a fair fraction of its purchase price.

12 Enemy System

Create at least **25 different enemies**, grouped by level range, with strength scaling as the player progresses.

Level Range	Example Enemies
1 – 10	Goblin, Wolf, Slime, Zombie, Skeleton
11 – 20	Orc, Bandit, Troll, Dark Archer, Giant Spider
21 – 30	Vampire, Werewolf, Necromancer, Ice Golem, Fire Demon
31 – 100	Continue the pattern — introduce new enemy families for each region (Desert, Ruins, Castle, Volcano, Frozen Mountain, Sky Temple, Demon Realm)

Enemy HP, attack, and defense should scale based on the player's current level so that combat remains balanced throughout the game.

13 Boss System

Every 10 player levels, a boss must appear. **The player cannot progress beyond the current level cap until that boss is defeated.**

Level	Boss
10	Goblin King
20	Forest Guardian
30	Ancient Golem
40	Vampire Lord
50	Dragon Rider
60	Demon General
70	Ice Titan
80	Shadow Emperor
90	Celestial Dragon
100	Ancient Demon King

Every boss should feel like a mini-event that requires preparation and strategy. Bosses must include:

- ✓ Very High HP
- ✓ Multiple Skills
- ✓ Critical Hits
- ✓ Healing Ability
- ✓ Special Attack every 3 turns
- ✓ Higher loot drop rates
- ✓ Guaranteed rare-or-better rewards

14 Leveling System

The player gains EXP after defeating enemies. Each level-up should increase HP, Mana, Attack, and Defense.

- ✓ Every 5 levels: unlock a new skill or passive bonus
- ✓ Every 10 levels: defeat the region boss to unlock the next region
- ✓ Maximum Level: 100

15 Loot Drop System

Defeated enemies should randomly drop Gold, Potions, Weapons, Armor, or Materials. Drop chances should scale inversely with rarity:

Rank	Drop Chance
Common	50%
Uncommon	25%
Rare	15%
Super Rare	6%
Epic	3%
Mythical	0.9%
Legendary	0.1%

16 World Map

Village → Forest → Cave → Desert → Ruins → Castle → Volcano → Frozen Mountain → Sky Temple → Demon Realm

Each region unlocks only after the player defeats that region's boss.

17 Menus and Game States

Main Menu

```
========================
LEGENDS OF THE FORGOTTEN REALM
========================
1. New Game
2. Continue
3. Instructions
4. Exit
```

Game Over

```
==================
GAME OVER
==================
1  Retry
2  Load Save
3  Exit
```

Victory Screen

After the Ancient Demon King is defeated, display a full end-game summary:

- ✓ Player Name
- ✓ Final Level
- ✓ Total Gold
- ✓ Bosses Defeated
- ✓ Enemies Defeated
- ✓ Weapons Collected
- ✓ Rare Items Found
- ✓ Total Play Time
- ✓ Final Score

18 Bonus Features (Optional)

Students may implement any of the following for extra credit:

Elemental Damage **Status Effects** **Dodge & Accuracy** **Weapon Durability** **Enchantments**
Crafting System **Daily Quests** **Random World Events** **NPC Dialogue** **Arena / Gambling Mode**
Multiple Save Slots **Difficulty Modes** **Colorama Terminal Colors** **ASCII Art** **Sound Effects**

19 Technical Requirements

Your solution should clearly demonstrate:

- ✓ Clean, modular code (multiple .py files recommended)
- ✓ Input validation and exception handling
- ✓ Meaningful variable and function names
- ✓ Comments where appropriate
- ✓ Use of Python's `random` and `json` modules
- ✓ Efficient use of lists, dictionaries, sets, and tuples

20 Submission Requirements

- ✓ Complete source code
- ✓ A README.md with setup and gameplay instructions
- ✓ A playable game from start to finish
- ✓ At least 25 enemy types
- ✓ At least 20 unique weapons across all rarity tiers
- ✓ At least 12 armor sets
- ✓ At least 15 potion types (health, mana, mixed, buff)
- ✓ Boss battles every 10 levels
- ✓ Save/Load functionality
- ✓ Clean code, proper formatting, and good UX

21 Evaluation Rubric

Criteria	Marks
Core Gameplay (Combat, Turns, Leveling)	20
Weapon, Armor & Rarity System	15
Boss Battles & Progression	15
Inventory, Shop & Potion System	10
Save/Load (JSON)	10
Python Concepts & Code Quality	15
Input Validation & Error Handling	5
User Interface & Gameplay Experience	5
Creativity & Bonus Features	5
Total	**100**

Note to students: This specification gives you a clear roadmap while leaving enough design freedom to build a unique RPG using procedural programming, OOP, or a hybrid approach. It scales naturally from early-game progression to challenging boss encounters every 10 levels. Focus on getting a clean, playable core loop working first — combat, leveling, inventory — before layering on bonus features.

"A hero is not measured by the sword they carry, but by the code that forges it."